

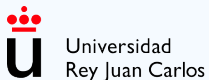
Big Data Processing II

Tema 2: Apache Kafka

Alberto Fernández Isabel
Natalia Madrueño Sierro
Rubén Rodríguez Fernández

DSLAB

Febrero 2026





- Entender qué problema resuelve en el **análisis de datos deportivos**
- Conocer la arquitectura y los **conceptos clave** de Kafka
- Identificar **casos de uso** en datos deportivos en tiempo real
- Aprender **buenas prácticas** de diseño y operación de Kafka



- 1 **Introducción**
- 2 **Componentes**
- 3 **Infraestructura**
- 4 **Ejemplos**



- 1 **Introducción**
- 2 Componentes
- 3 Infraestructura
- 4 Ejemplos



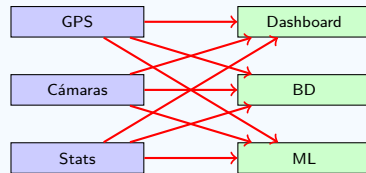
- **Con pocas integraciones todo es simple**
 - Antes $\Rightarrow$ una aplicación alimenta directamente a otra
 - Ahora $\Rightarrow$ múltiples fuentes y múltiples destinos
- **La integración punto a punto introduce deuda técnica**
 - Cada fuente habla con cada destino
 - Formatos y contratos inconsistentes
 - Difícil rastrear el flujo de información end-to-end
 - Un cambio en un productor rompe consumidores en cascada
- **La complejidad crece rápidamente con cada nueva integración**

- **Sistema de datos en un estadio**

- Sensores GPS de jugadores
- Cámaras de tracking
- Sistema de estadísticas en vivo

- **El resultado es caos y complejidad**

- Acoplamiento fuerte entre sistemas
- n fuentes $\times$ m destinos = $n \times m$ integraciones



9 conexiones para 6 sistemas



- **Introducir un intermediario central**

- Productor → Distribuidor → Consumidor

- **Hub central de datos deportivos**

- Productores → clasifican y publican eventos
- Consumidores → se suscriben a lo que necesitan

- **Menos acoplamiento y mayor flexibilidad**

- Añadir consumidores no implica cambiar productores y viceversa
- Permite múltiples consumidores con diferentes necesidades

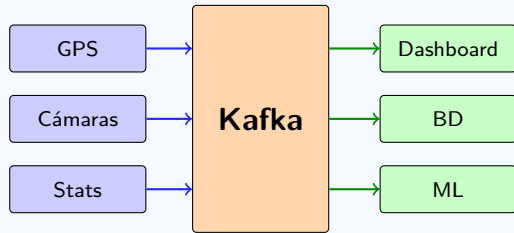


• Kafka como hub central de mensajería

- Todas las fuentes envían a Kafka
- Todas las herramientas leen de Kafka
- Los distribuye a múltiples destinos

• Desacoplamiento entre sistemas

- $n + m$ integraciones vs $n \times m$
- Sistemas independientes
- Formato estandarizado
- Escalabilidad horizontal



6 conexiones para 6 sistemas



- **Colas tradicionales**

- Diseñadas para distribución de tareas
- Mensajes eliminados tras consumo
- Un consumidor por mensaje

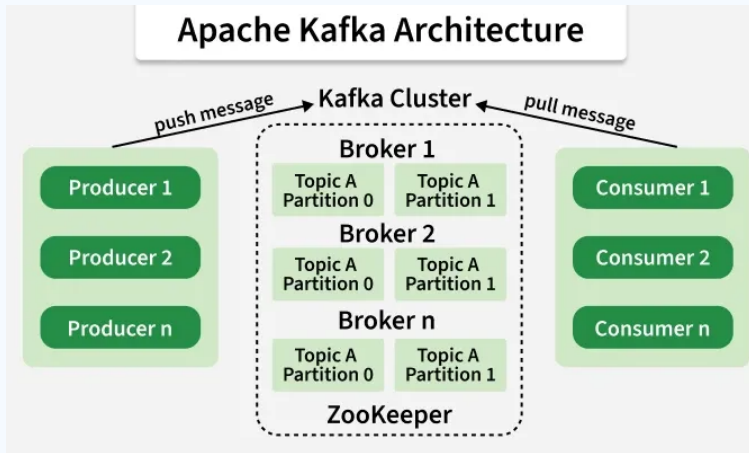
- **Logs persistentes**

- Diseñadas para análisis de datos y eventos
- Mensajes persistentes con retención configurable
- Múltiples consumidores independientes
- Replay de eventos históricos

- **Kafka es un commit log distribuido**



- **Desacopla fuentes (productores) de herramientas (consumidores)**
- **Múltiples consumidores para el mismo flujo de datos**
 - Dashboard en vivo para entrenadores
 - Análisis táctico con machine learning
 - Almacenamiento histórico en base de datos
- **Facilita análisis de datos deportivos en vivo**
 - Posiciones GPS de jugadores cada segundo
 - Eventos del juego, como goles, pases, tiros, tarjetas
 - Métricas fisiológicas, como ritmo cardíaco, velocidad



<https://www.geeksforgeeks.org/apache-kafka/kafka-architecture/>



- 1 Introducción
- 2 Componentes**
- 3 Infraestructura
- 4 Ejemplos

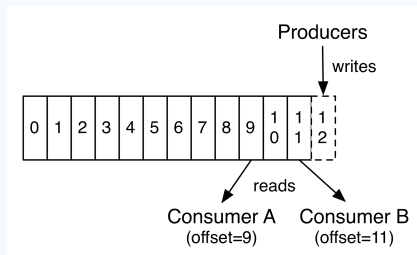


• Estructura append-only

- Escrituras secuenciales al final
- Inmutabilidad de eventos pasados
- Alta eficiencia de escritura y lectura
- Durabilidad de eventos configurable

• Offsets como punteros de lectura

- Identificador único por mensaje en partición
- Orden secuencial de eventos por partición
- Permite lectura determinista por partición
- Checkpointing del progreso del consumidor





- **Estructura de un mensaje**

- **Key** (opcional) → ayuda a decidir partición y mantener orden por entidad
- **Value** → payload del mensaje en bytes
- **Timestamp** → marca temporal (por defecto del productor)

- **Kafka no impone un formato específico**

- JSON (recomendado)
- Apache Avro para consistencia



- Ejemplo de evento para gol en un partido

```
// Kafka Key liga_2025_j27_001, Timestamp 2025-03-15T16:42:18Z
{
  "schema_version": "1.0",
  "event_id": "3f7a2b9c-5d8e-4c1f-a6b3-9e4d2c1b5a7f",
  "event_time": "2025-03-15T16:42:18Z",
  "match_id": "liga_2025_j27_001",
  "event_type": "goal",
  "team_id": "real_madrid",
  "opponent_id": "barcelona",
  "scorer_id": "vinicius_jr",
  "assister_id": "modric",
  "minute": 67
}
```



- **Contrato que define la estructura de los datos**

- Plantilla que define qué campos tiene cada evento
- Facilita entendimiento entre productores y consumidores
- Evita errores → si falta un campo se detecta inmediatamente

- **Ejemplo de esquema para evento de gol**

- `event_id: string`
- `event_time: timestamp`
- `match_id: string`
- `event_type: string`
- `scorer_id: string`
- `assister_id: string`
- `minute: integer`



- **Lotes de mensajes**

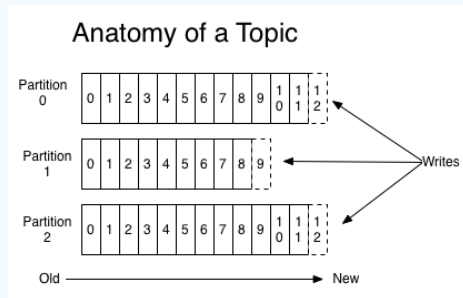
- Colección de mensajes del mismo Topic y partición
- Reduce overhead de red y permite compresión
- En lugar de enviar 1 evento a la vez envía varios juntos
- Trade-off → lotes grandes pueden incrementar la latencia

- **Compresión de datos**

- Ahorra espacio y velocidad de red
- Kafka soporta compresión configurable
- Existen varios algoritmos de compresión



- **Canales lógicos donde se organizan los mensajes**
 - Categoría de mensajes relacionados
 - Organiza eventos por tipo o dominio
 - Patrón común de nombrado → `dominio.subdominio.tipo.version`
- **Configuraciones clave**
 - Número particiones para paralelismo
 - Factor de replicación para durabilidad
 - Tiempo de retención de los mensajes
- **Ejemplo** `olympics.swimming.events.v1`



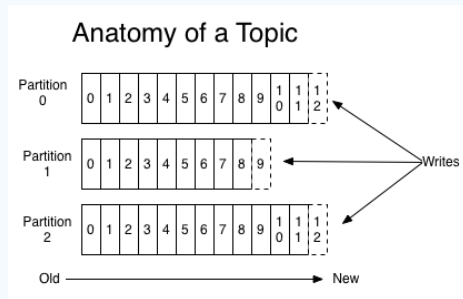


● Procesamiento en paralelo

- Un Topic se divide en múltiples particiones
- Unidad de paralelismo y escalado horizontal
- Cada partición es un log ordenado independiente
- Es *append-only* → se escribe al final

● Orden garantizado dentro de una partición

- No garantiza orden en todo el Topic
- Key → misma partición
- Sin key → round-robin o sticky





- **Productores**

- Escriben mensajes en Topics
- Balancean carga entre particiones
- Con *key* garantizan orden por entidad en una partición

- **Consumidores**

- Leen mensajes manteniendo orden dentro de cada partición
- Mantienen offsets con la posición del último mensaje leído
- Para recuperación tras fallo reinician desde el último offset guardado



- **Marcadores de lectura en partición**

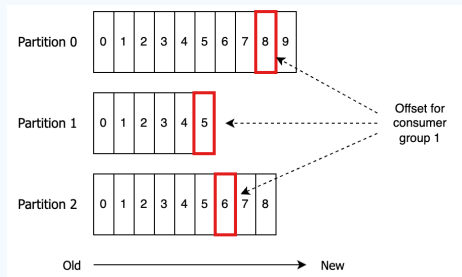
- Permite recordar qué eventos se han procesado
- Como un marcador en un libro

- **Ejemplo práctico de uso de offsets**

- Se disponen 1000 eventos GPS
- La lectura se cae en el evento 750
- Al reiniciar se continúa desde el 751

- **Commit automático vs manual**

- En *Auto* → Guarda el offset automáticamente
- En *Manual* → Tú decides cuándo guardarlo





- **Indica qué tan atrasado está el consumidor**

- Diferencia entre ultimo offset disponible y offset del consumidor
- End Offset=1000, Current Offset=750 → Lag=250 mensajes

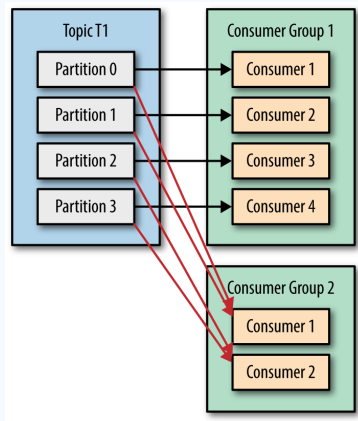
- **Importancia de monitorear el lag**

- Lag alto → consumidor lento o problemas de procesamiento
- Lag creciente → consumidor no puede mantener el ritmo
- Alertar cuando supera umbrales críticos



- **Un Topic puede ser leído por múltiples consumidores**
- **Mismo grupo de consumidores**
 - Múltiples consumidores procesando en paralelo un Topic
 - Si un consumidor procesa rápido, reduce el lag del grupo
 - Si un consumidor falla, otros continúan
 - Ejemplo → 3 consumidores procesando eventos GPS
- **Único consumidor (distinto grupo)**
 - Aplicaciones independientes
 - Ejemplo → Dashboard + Análisis ML + DB histórica

- **Permite paralelizar la lectura de un Topic**
 - Escalabilidad horizontal de consumidores
 - Consumidores agrupados por `group_id`
 - Paraleliza lectura dentro de un Topic
- **Los mensajes se distribuyen en particiones**
 - Dentro del grupo cada partición la lee un solo consumidor a la vez
 - Para rendimiento máximo de lectura → particiones $\approx$ consumidores (en el grupo)
- **Si un consumidor falla los restantes se reparten sus particiones**





- **Reasignación automática de particiones**
 - Se activa cuando un consumidor se une, sale o falla
 - Las particiones se redistribuyen entre consumidores activos
- **Impacto durante el rebalance**
 - El grupo deja de consumir temporalmente
 - Puede causar duplicados si no se commitean offsets antes
 - Tiempo de pausa depende del tamaño del grupo
- **Varias estrategias de asignación**
 - Range, Round-robin, Sticky



- 1 Introducción
- 2 Componentes
- 3 Infraestructura**
- 4 Ejemplos



- **Broker**

- Almacena particiones asignadas
- Es un servidor individual (nodo) del clúster
- Tienen un identificador único

- **Clúster**

- Conjunto de brokers coordinados
- Un broker actúa como controlador
- Distribución de particiones entre brokers

- **Permite escalabilidad y alta disponibilidad**

- **Modelo líder-seguidor**

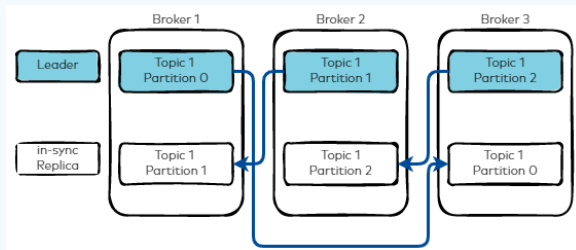
- Líder maneja lecturas y escrituras
- Seguidores copian log del líder

- **In-Sync Replicas (ISR)**

- Réplicas sincronizadas con el líder
- Pueden ser elegidas como líder

- **Si el líder falla se elige otro**

- Nuevo líder es un seguidor en ISR
- El sistema sigue funcionando sin pérdida





- **Acknowledgment (ack)**

- Recibido por el productor
- Brokers confirman mensajes recibidos

- **Tipos de acknowledgment**

- `acks=0` → sin confirmación
- `acks=1` → líder confirma
- `acks=all` → todas las ISR confirman



- **At-most-once**

- El dato puede perderse pero no se duplica
- Cuando velocidad es más importante que completitud
- Ejemplo → Sensor GPS falla, pierdes una posición

- **At-least-once** - Opción más común

- El dato llega pero puede duplicarse
- Garantiza que no se pierden eventos
- Ejemplo → El mismo gol aparece dos veces en tu sistema

- **Exactly-once**

- El dato se procesa una vez
- Kafka lo soporta pero requiere configuración avanzada



- **Durabilidad es no perder datos**
- **Tres pilares para garantizar durabilidad**
 - Producers esperan confirmación del broker
 - Múltiples copias en ISR
 - Datos escritos a disco
- **Mayor durabilidad implica mayor latencia**

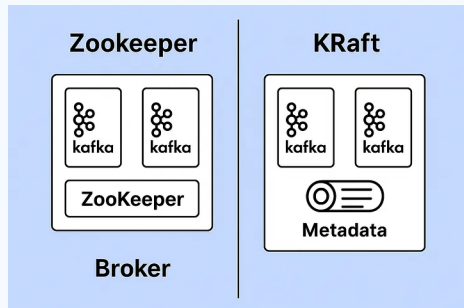
- **Zookeeper (legacy)**

- Almacena metadatos del clúster y coordina elección de líderes
- Ensemble con número impar de nodos (3, 5, 7)
- Requiere gestión adicional de infraestructura

- **KRaft (Kafka 3.0+)**

- Gestiona metadatos y líderes sin Zookeeper
- Protocolo Raft para consenso y elección de líderes
- Menos complejidad y arranque más rápido

- **Se recomienda KRaft para nuevos clústeres**





- **Regla práctica**

- Particiones $\approx$ **throughput deseado** / **velocidad del consumidor**
- Ejemplo $\rightarrow 1 \text{ GB/s} / 50 \text{ MB/s} \approx 20$ particiones

- **Consideraciones**

- Se pueden aumentar, pero no disminuir
- Más particiones $\rightarrow$ más concurrencia
- Demasiadas $\rightarrow$ mayor uso de memoria y coste de coordinación



- **Retención por tiempo**

- Se borran segmentos cuando el último mensaje supera la edad límite
- Ejemplo → conservar datos de un partido durante 7 días

- **Retención por tamaño**

- Útil para evitar llenar el disco con flujos impredecibles
- Ejemplo → máximo 1GB por partición

- **Log compaction**

- Mantiene el último valor por clave
- Elimina versiones antiguas de la misma clave
- Ejemplo → Mantener última posición GPS de cada jugador



- 1 Introducción
- 2 Componentes
- 3 Infraestructura
- 4 Ejemplos**



- **Kafka 4.1+ incluye KRaft (sin Zookeeper)**
 - Arranque rápido con un solo contenedor
 - Ideal para desarrollo y pruebas locales
- **Comando para lanzar Kafka localmente**
 - Puerto 9092 expuesto para productores y consumidores
 - Broker disponible en localhost:9092

>_ Shell Terminal

```
docker run -p 9092:9092 apache/kafka:4.1.1
```



>_ Shell Terminal

```
docker run -p 9092:9092 \
  -e KAFKA_HEAP_OPTS="-Xmx1G -Xms1G" \           # Memoria JVM
  -e KAFKA_LOG_RETENTION_HOURS=168 \              # Retencion 7 dias
  -e KAFKA_NUM_PARTITIONS=3 \                     # Particiones defecto
  -e KAFKA_DEFAULT_REPLICATION_FACTOR=1 \         # Factor replicacion
  -e KAFKA_COMPRESSION_TYPE=gzip \                # Compresion
  apache/kafka:4.1.1
```



```
services:
  kafka:
    image: apache/kafka:4.1.1
    ports:
      - "9092:9092"
    volumes:
      - kafka-data:/var/lib/kafka/data      # Persistencia
    environment:
      KAFKA_HEAP_OPTS: "-Xmx1G -Xms1G"      # Memoria JVM
      KAFKA_LOG_RETENTION_HOURS: 168         # Retencion 7 dias
      KAFKA_NUM_PARTITIONS: 3                # Particiones defecto
      KAFKA_DEFAULT_REPLICATION_FACTOR: 1    # Factor replicacion
      KAFKA_COMPRESSION_TYPE: gzip           # Compresion
```



- **Librería kafka-python para casos de uso sencillos**
 - Cliente básico de Kafka para Python
 - Soporte completo para Producer y Consumer
- **Componentes principales**
 - `KafkaProducer`: Envía mensajes a topics
 - `KafkaConsumer`: Lee mensajes de topics
 - `KafkaAdminClient`: Gestión de topics y configuración
- **Librería confluent-kafka para producción y alto rendimiento**



Python Code

```
from kafka import KafkaProducer
import json
import random
import time

producer = KafkaProducer(
    bootstrap_servers=['localhost:9092'],
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

for i in range(100):
    message = {'id': i, 'value': random.randint(1, 100)}
    producer.send('random.numbers.v1', value=message)
    time.sleep(1)

producer.close()
```



Python Code

```
producer = KafkaProducer(  
    bootstrap_servers=['localhost:9092'],  
    value_serializer=lambda v: json.dumps(v).encode('utf-8'),  
  
    # Garantías de entrega  
    acks='all',           # 0, 1, o 'all'  
    retries=3,            # reintentos  
    enable_idempotence=True, # exactly-once  
  
    # Rendimiento  
    compression_type='gzip', # gzip, snappy, lz4, zstd  
    batch_size=16384,        # tamaño lote (bytes)  
    linger_ms=10             # espera antes de enviar  
)
```



Python Code

```
from kafka import KafkaConsumer
import json

consumer = KafkaConsumer(
    'random.numbers.v1',
    bootstrap_servers=['localhost:9092'],
    auto_offset_reset='latest',
    value_deserializer=lambda m: json.loads(m.decode('utf-8'))
)

print("Listening for messages...")

for message in consumer:
    data = message.value
    print(f"Received: ID={data['id']}, Value={data['value']}")
```



Python Code

```
consumer = KafkaConsumer(  
    'random.numbers.v1',  
    bootstrap_servers=['localhost:9092'],  
  
    # Deserialización de mensajes  
    key_deserializer=lambda k: k.decode('utf-8') if k else None,  
    value_deserializer=lambda m: json.loads(m.decode('utf-8')),  
  
    # Grupo de consumidores  
    group_id='my-consumer-group',  
  
    # Gestión de offsets  
    auto_offset_reset='latest',    # earliest, latest, none  
)
```



- **Kafka como hub central de datos deportivos**

- Reduce complejidad y fragilidad frente a integraciones P2P
- Hub central para datos deportivos en tiempo real

- **Kafka es un commit log distribuido**

- Persistencia y retención → replay y múltiples consumidores
- Topics/particiones → orden por partición y paralelismo
- Offsets → control del progreso y recuperación ante fallos

- **Operación y escalado requieren diseño**

- Replicación líder/seguidores, dimensionamiento de particiones
- Políticas de retención según tipo de dato deportivo
- Configuración apropiada para garantías de entrega